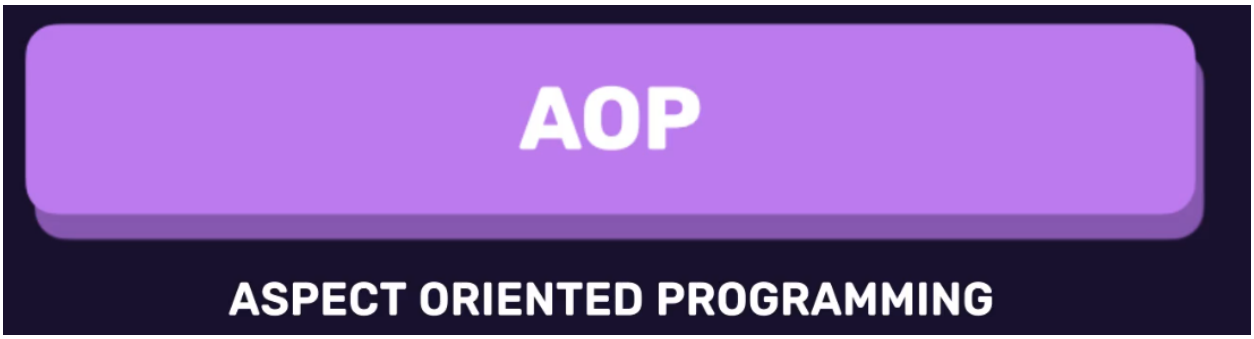
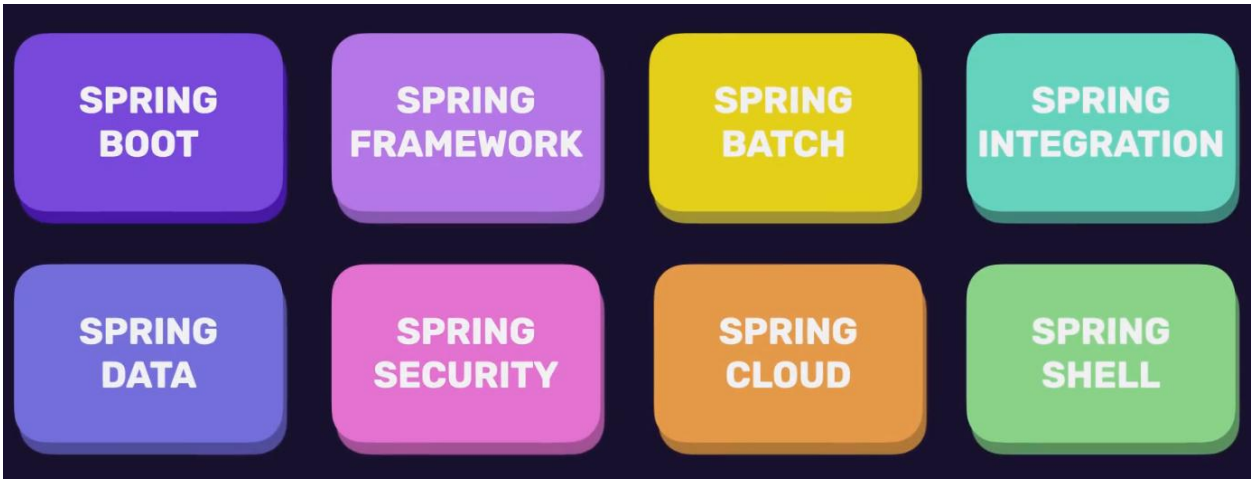
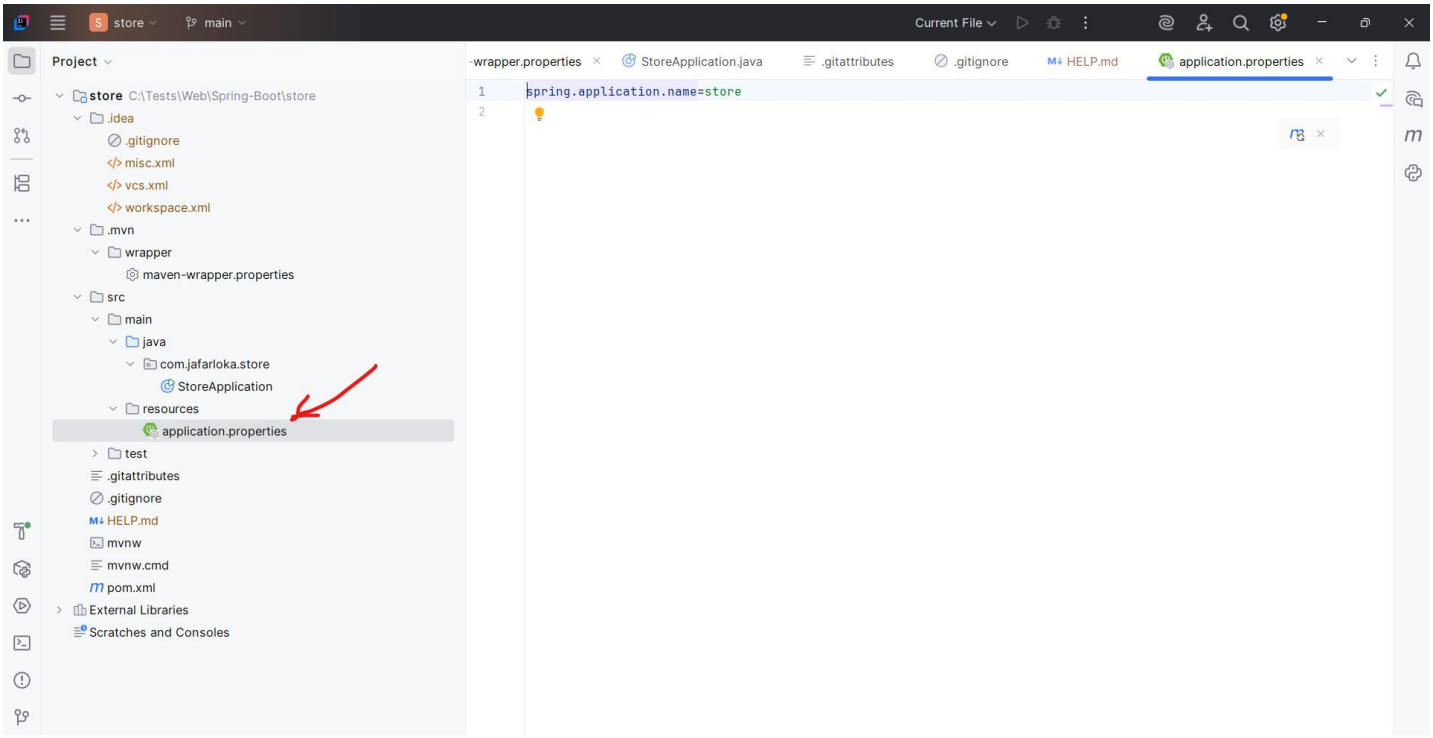
AOP used for adding features that doesn't exists in framework (to extend it).



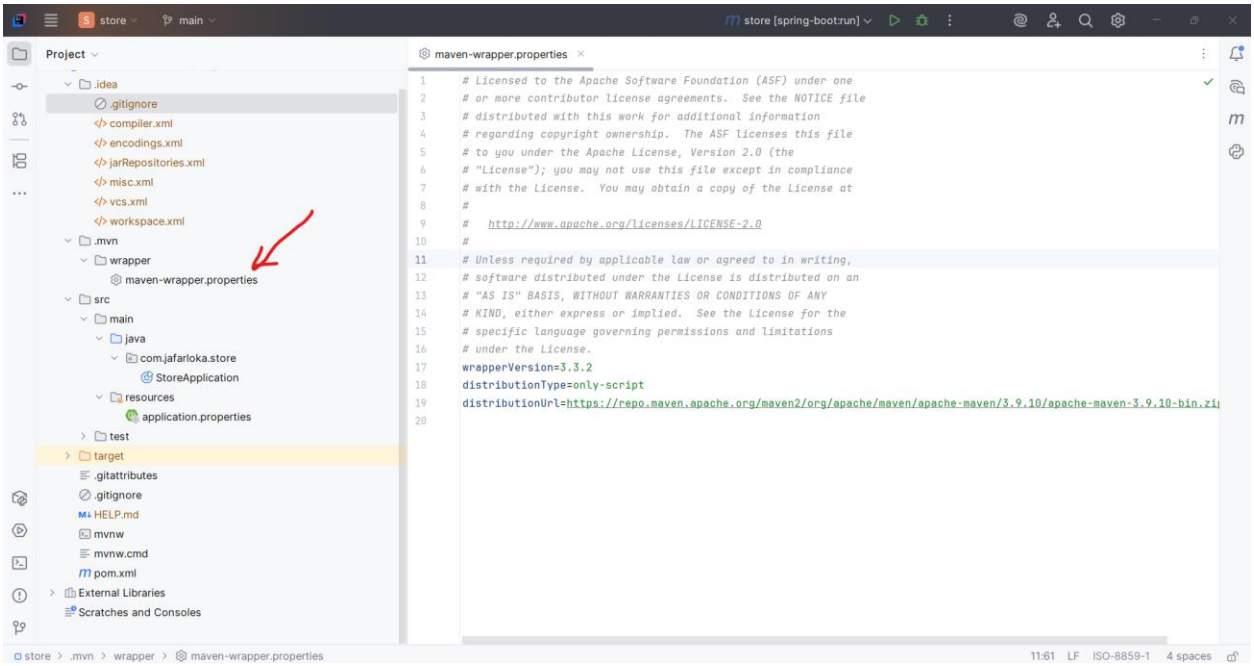
****



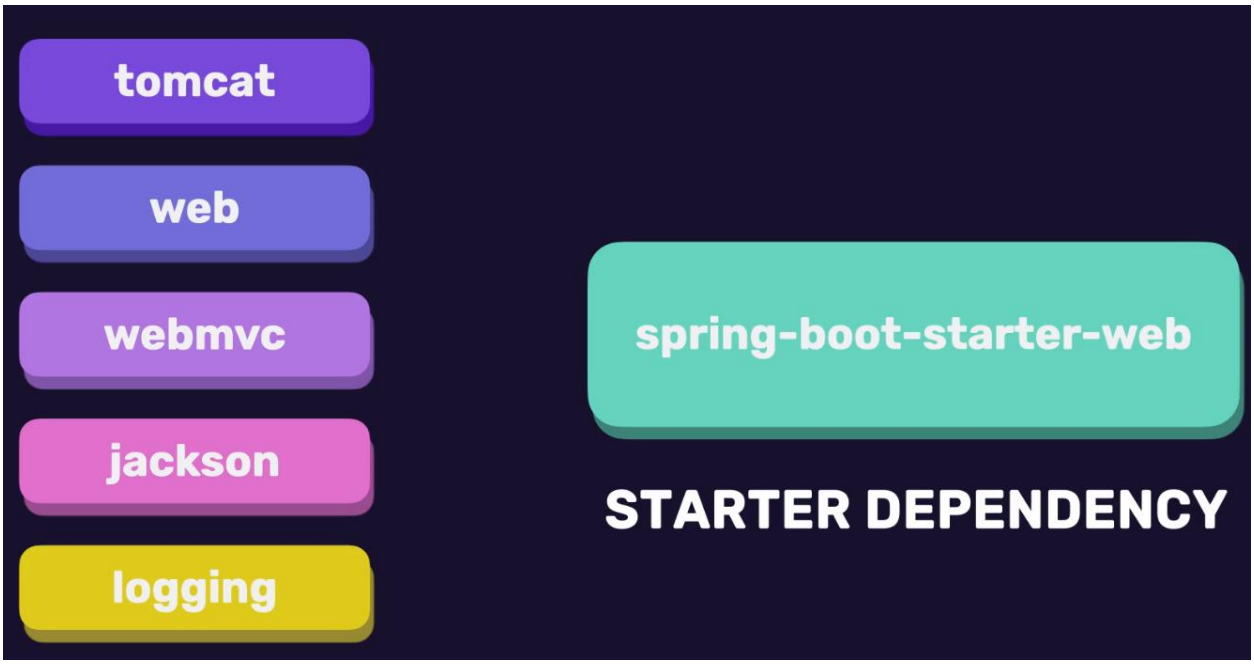
****


store
main
Current File
Project
store C:\Tests\Web\Spring-Boot\store
.idea
.gitignore
misc.xml
vcs.xml
workspace.xml
.mvn
wrapper
maven-wrapper.properties
src
main
java
com.jafarloka.store
StoreApplication
resources
application.properties
test
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
External Libraries
Scratches and Consoles
-wrapper.properties
StoreApplication.java
.gitattributes
.gitignore
HELP.md
application.properties
1 spring.application.name=store
2

****


store
main
store [spring-boot:run]
Project
maven-wrapper.properties
.idea
.gitignore
compiler.xml
encodings.xml
jarRepositories.xml
misc.xml
vcs.xml
workspace.xml
.mvn
wrapper
maven-wrapper.properties
src
main
java
com.jafarloka.store
StoreApplication
resources
application.properties
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
External Libraries
Scratches and Consoles
Licensed to the Apache Software Foundation (ASF) under one
or more contributor license agreements. See the NOTICE file
distributed with this work for additional information
regarding copyright ownership. The ASF licenses this file
to you under the Apache License, Version 2.0 (the
"License"); you may not use this file except in compliance
with the License. You may obtain a copy of the License at
#
http://www.apache.org/licenses/LICENSE-2.0
#
Unless required by applicable law or agreed to in writing,
software distributed under the License is distributed on an
"AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
KIND, either express or implied. See the License for the
specific language governing permissions and limitations
under the License.
wrapperVersion=3.3.2
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.10/apache-maven-3.9.10-bin.zi
store > .mvn > wrapper > maven-wrapper.properties
11:61 LF ISO-8859-1 4 spaces

****



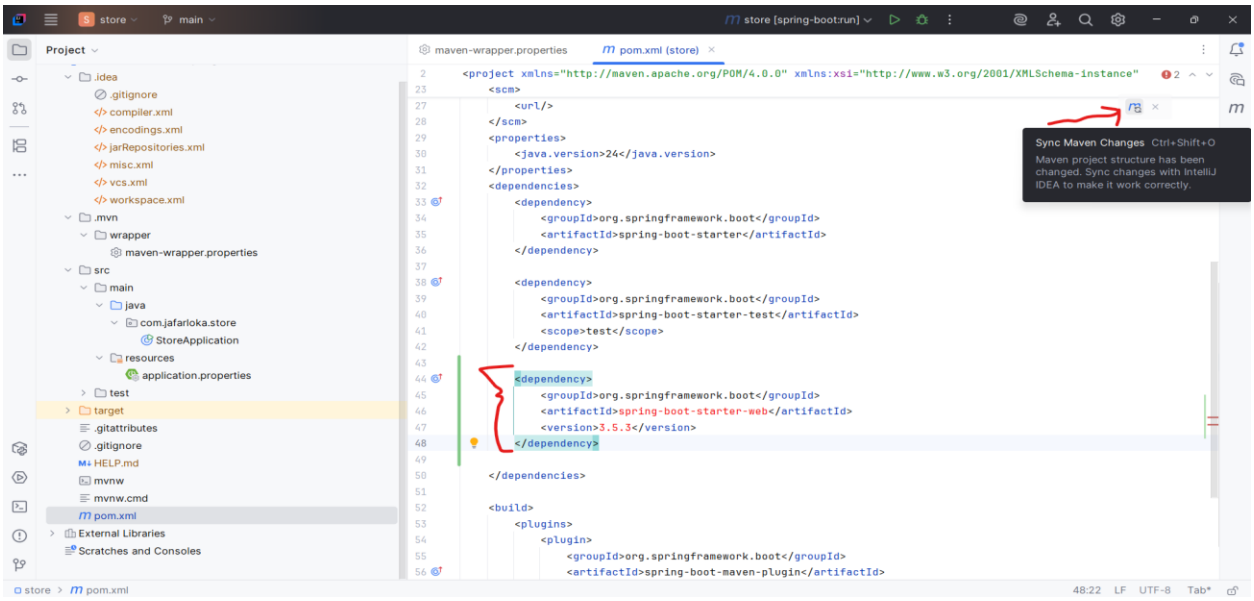
****

Maven Central: https://central.sonatype.com/

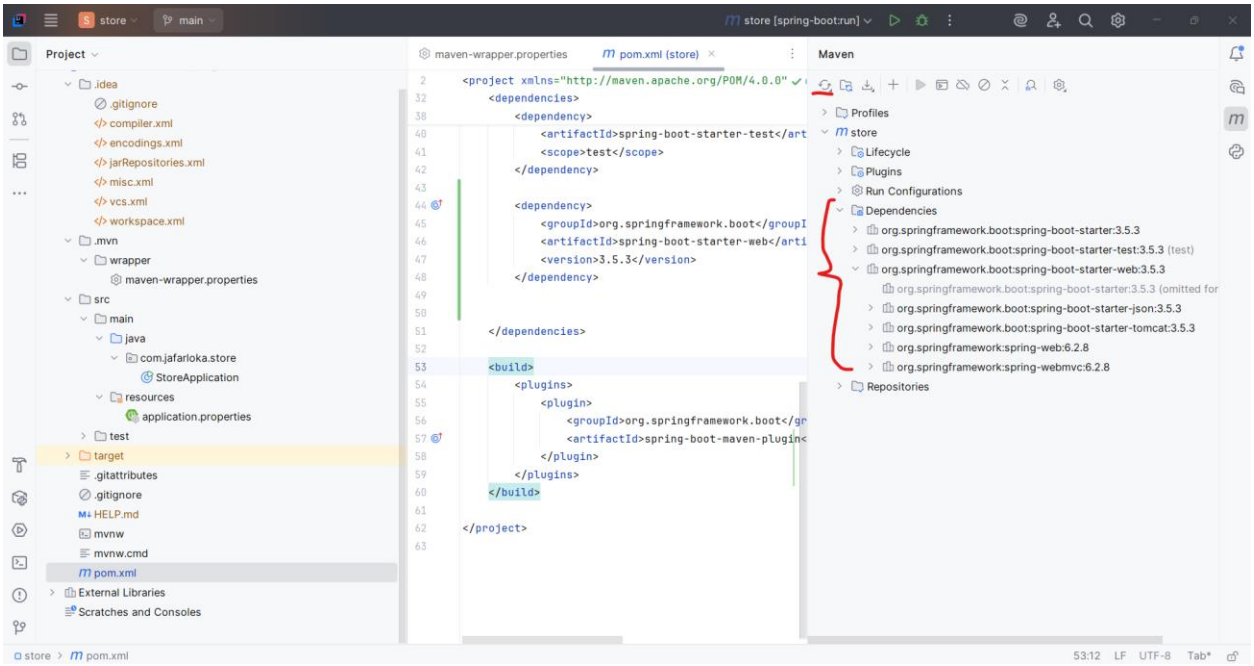
Here we can search for dependencies like **spring-boot-starter-web**

****

After adding the dependency, it may not be found on the local machine, so we must download it:


store
main
store [spring-boot:run]
Project
maven-wrapper.properties
pom.xml (store)
Sync Maven Changes Ctrl+Shift+O
Maven project structure has been changed. Sync changes with IntelliJ IDEA to make it work correctly.
store > pom.xml
48:22 LF UTF-8 Tab*

****


store
main
store [spring-boot:run]
Project
maven-wrapper.properties
pom.xml (store)
Maven
Profiles
store
Lifecycle
Plugins
Run Configurations
Dependencies
org.springframework.boot:spring-boot-starter:3.5.3
org.springframework.boot:spring-boot-starter-test:3.5.3 (test)
org.springframework.boot:spring-boot-starter-web:3.5.3
org.springframework.boot:spring-boot-starter:3.5.3 (omitted for
org.springframework.boot:spring-boot-starter-json:3.5.3
org.springframework.boot:spring-boot-starter-tomcat:3.5.3
org.springframework:spring-web:6.2.8
org.springframework:spring-webmvc:6.2.8
Repositories
store > pom.xml
53:12 LF UTF-8 Tab*

****

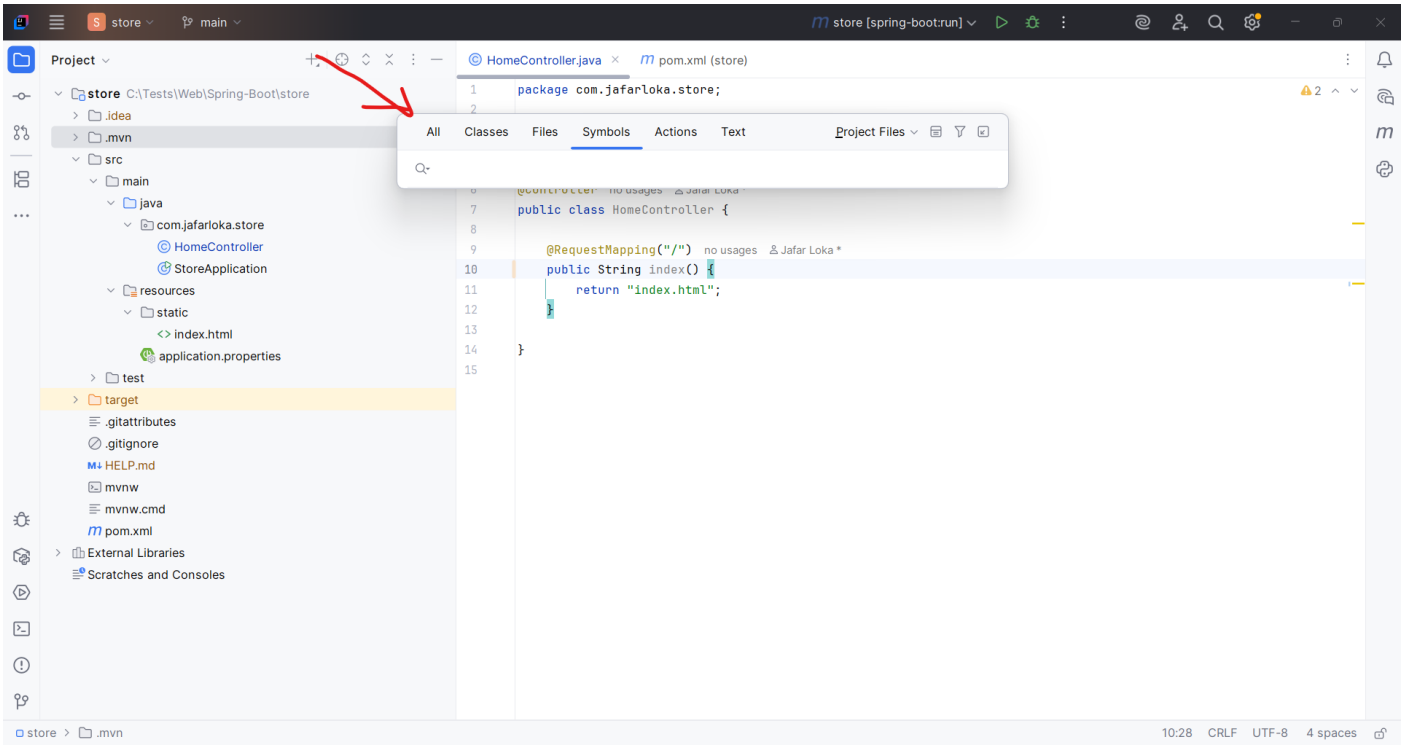
We can run a Spring Boot project using: *mvnw spring-boot:run*

OR from:



****

SHIFT + CTRL + O → For searching for file in Intellij IDE



****

ALT + 0 → Generate dependency

****

To define specific configuration, we go to application.properties:

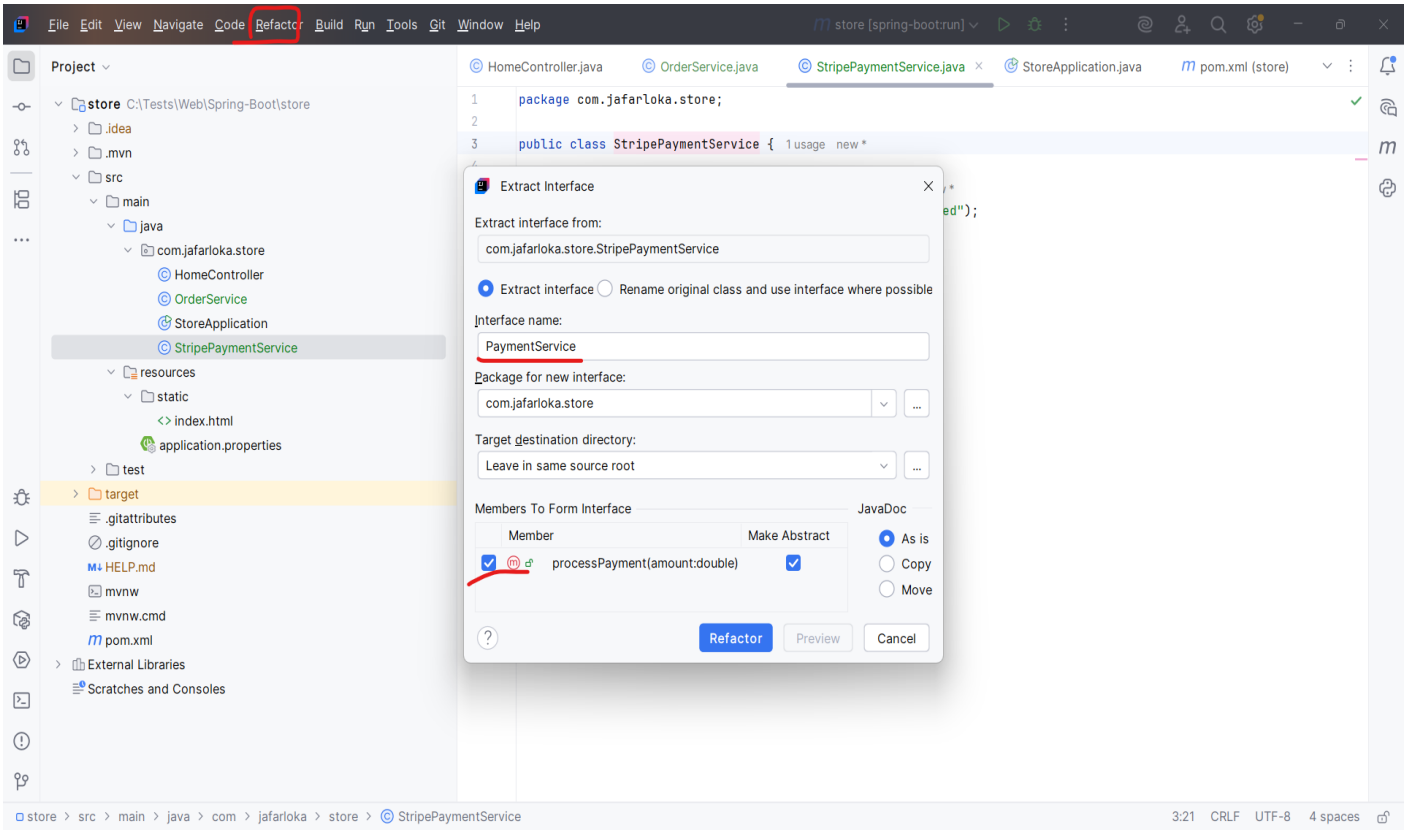
```
spring.application.name=store
jloka.value=100
```

Then to use their value on code, we use Value-annotation, with ${}:

```
@Value("${spring.application.name}")
private String appName;

@Value("${jloka.value}")
private String value;
```

****

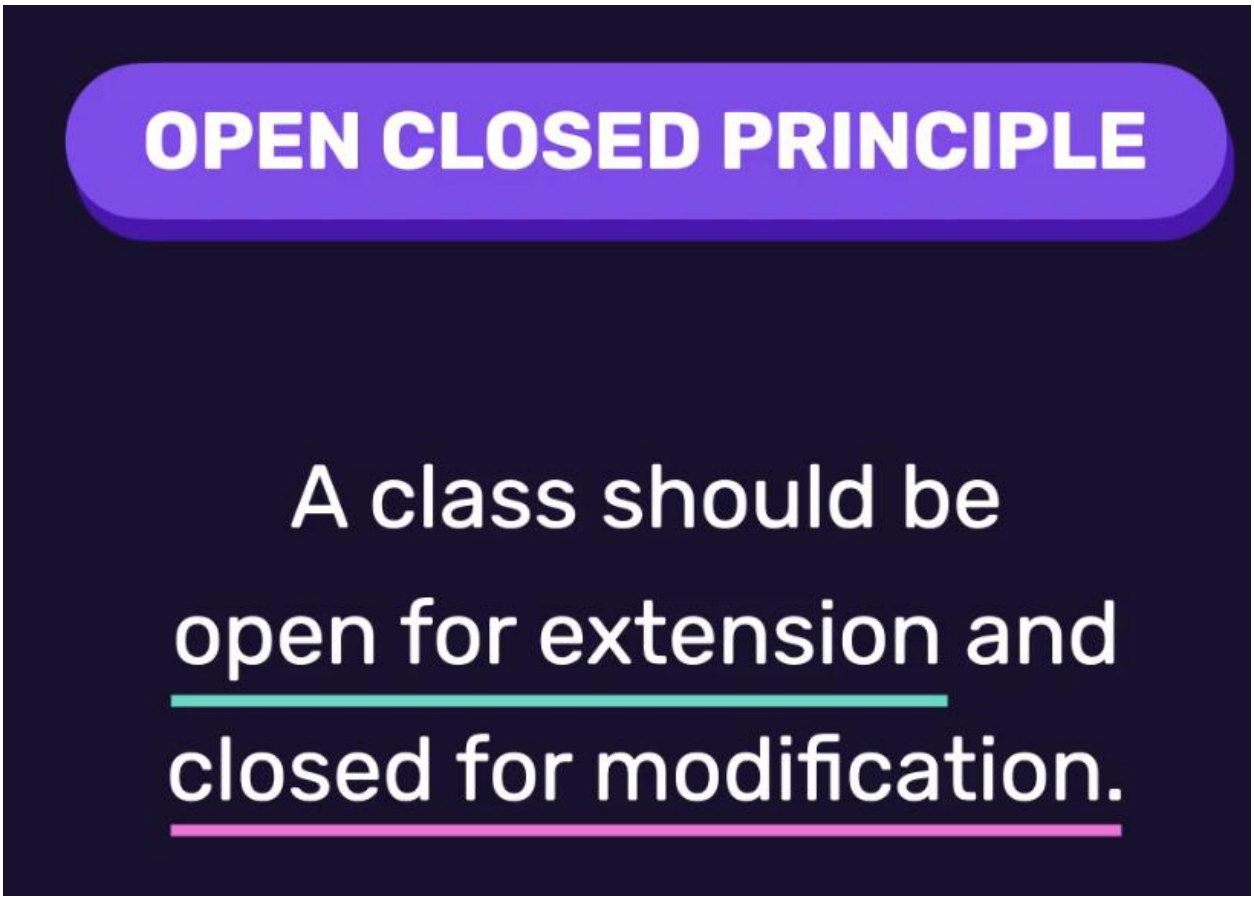


****

Note: tight couples mean we have bad service coupling with other, may it can be in small projects, or in projects that have less features in some part of them.

****

From solid principle, And OOP:



****

We can use DI using setter-methods:

```
public class OrderService {

    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
    public void placeOrder() {
        paymentService.processPayment(100_000);
    }

}
```

****

We can use Generate from Intellij to create the setter methods, or other tasks:

```
public class OrderService {  3 usages   Jafar Loka *

    private PaymentService paymentService;  3 usages

    public OrderService(PaymentService                .paymentService = paymentService; }

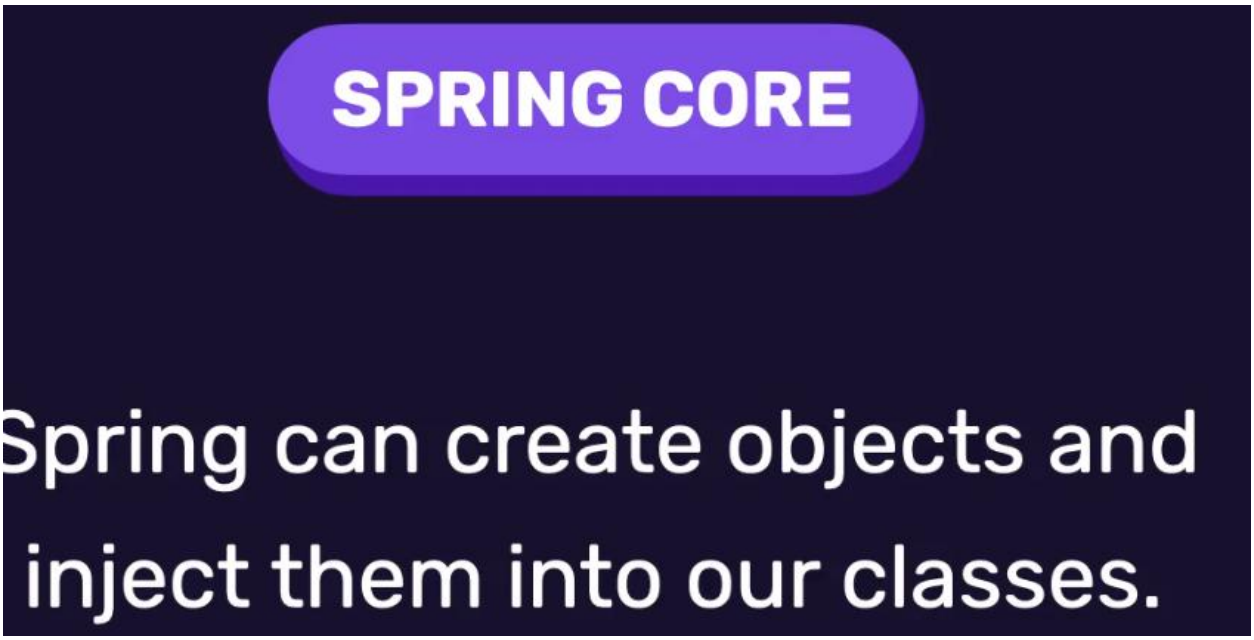
    public void setPaymentService(Paym                ce) {  2 usages   new *
        this.paymentService = paymentS
    }
```

Generate

- Constructor
- Logger
- Getter
- Setter
- Getter and Setter
- equals() and hashCode()
- toString()
- Override Methods...
- Delegate Methods...
- Test...
- Copyright

```
4:06:12.313+03:00  INFO 13372 --- [stor                a.c.c.C.[Tomcat].[localhost].[/]
4:06:12.314+03:00  INFO 13372 --- [stor                s.c.ServletWebServerApplicationContext
4:06:12.371+03:00  INFO 13372 --- [stor                s.b.a.w.s.WelcomePageHandlerMapping
4:06:12.533+03:00  INFO 13372 --- [stor                s.b.w.embedded.tomcat.TomcatWebServer
4:06:12.537+03:00  INFO 13372 --- [stor                m.jafarloka.store.StoreApplication
4:06:16.342+03:00  INFO 13372 --- [store] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]
```

****



****

IOC Container: set of objects that can be used for DI, each object called **BEAN**

Bean: Regular java code that used by spring core.

IOC: Inversion Of Control → Invert the control of *creating objects*, and *managing dependencies*.



****

To make IOC control the DI of OrderService class:

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

@SpringBootApplication
public class StoreApplication {

    public static void main(String[] args) {
       ApplicationContext context =
SpringApplication.run(StoreApplication.class, args);

       context.getBean(OrderService.class);
    }

}
```

To manage beans inside code:

